

ALGORITMA DAN STRUKTUR DATA
LAPORAN UAS NOMER GANJIL



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

UNTUK NOMER GANJIL

1. Buatlah program untuk pengurutan insertion secara ascending, dengan pengurutan dimulai dari nilai terbesar (sebelah kanan) sampai nilai terkecil (sebelah kiri).

Jawaban:

Kode:

```
1. #include <stdio.h>
2.
3. int main()
4. {
5.     int n;
6.     printf("masukkan jumlah elemen: ");
7.     if (scanf("%d", &n) != 1 || n <= 0)
8.     {
9.         printf("jumlah elemen harus positif.\n");
10.    } else {
11.        int arr[n];
12.        for (int i = 0; i < n; i++)
13.        {
14.            printf("masukkan elemen ke-%d: ", i + 1);
15.            scanf("%d", &arr[i]);
16.        }
17.
18.        for (int i = n - 2; i >= 0; i--)
19.        {
20.            int key = arr[i];
21.            int j = i + 1;
22.
23.            while (j < n && arr[j] < key)
24.            {
25.                arr[j - 1] = arr[j];
26.                j++;
27.            }
28.
29.            arr[j - 1] = key;
30.        }
31.
32.        printf("\nhasil pengurutan ascending: \n");
33.        for (int i = 0; i < n; i++)
34.        {
35.            printf("%d ", arr[i]);
```

```

36.     }
37.     printf("\n");
38. };
39.
40.     return 0;
41. }

```

Output:

```

masukkan jumlah elemen: 7
masukkan elemen ke-1: 2
masukkan elemen ke-2: 6
masukkan elemen ke-3: 8
masukkan elemen ke-4: 2
masukkan elemen ke-5: 5
masukkan elemen ke-6: 9
masukkan elemen ke-7: 1

hasil pengurutan ascending:
1 2 2 5 6 8 9

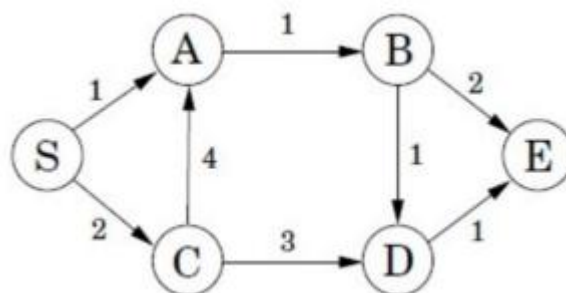
```

```

masukkan jumlah elemen: -14
jumlah elemen harus positif.

```

2. Dari graph berikut ini, buatlah jalur terpendek menggunakan algoritma Dijkstra dari titik awal ke titik akhir (ditentukan sendiri). Tampilkan rutenya dan beban minimalnya. Titik S=1, A=2, B=3, C=4, D=5, E=6.



Jawaban:

Kode:

```

1. #include <stdio.h>
2. #include <limits.h>
3.
4. #define V 6
5. #define INF INT_MAX
6.
7. int minDistance(int dist[], int visited[]);

```

```

8. void printPath(int parent[], int j, char names[]);
9. void dijkstra(int graph[V][V], int src, int dst, char
   names[]);
10.
11. int main()
12. {
13.     int graph[V][V] = {
14.         { 0, 1, 0, 2, 0, 0 },
15.         { 0, 0, 1, 4, 0, 0 },
16.         { 0, 0, 0, 0, 1, 2 },
17.         { 0, 0, 0, 0, 3, 0 },
18.         { 0, 0, 0, 0, 0, 1 },
19.         { 0, 0, 0, 0, 0, 0 }
20.     };
21.
22.     char names[V] = { 'S', 'A', 'B', 'C', 'D', 'E' };
23.
24.     int src = 0;
25.     int dst = 4;
26.
27.     dijkstra(graph, src, dst, names);
28.
29.     return 0;
30. }
31.
32. int minDistance(int dist[], int visited[])
33. {
34.     int min = INF;
35.     int min_index = -1;
36.
37.     for (int v = 0; v < V; v++)
38.     {
39.         if (!visited[v] && dist[v] <= min)
40.         {
41.             min = dist[v];
42.             min_index = v;
43.         }
44.     }
45.
46.     return min_index;
47. }
48.
49. void printPath(int parent[], int j, char names[])

```

```

50.{
51.    if (parent[j] == -1)
52.    {
53.        printf("%c", names[j]);
54.        return;
55.    }
56.
57.    printPath(parent, parent[j], names);
58.    printf(" -> %c", names[j]);
59.}
60.
61.void dijkstra(int graph[V][V], int src, int dst, char
    names[])
62.{
63.    int dist[V];
64.    int visited[V];
65.    int parent[V];
66.
67.    for (int i = 0; i < V; i++)
68.    {
69.        dist[i] = INF;
70.        visited[i] = 0;
71.        parent[i] = -1;
72.    }
73.
74.    dist[src] = 0;
75.
76.    for (int count = 0; count < V - 1; count++)
77.    {
78.        int u = minDistance(dist, visited);
79.
80.        if (u == -1)
81.        {
82.            break;
83.        }
84.
85.        visited[u] = 1;
86.
87.        for (int v = 0; v < V; v++)
88.        {
89.            if (!visited[v] && graph[u][v] && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v])
90.            {

```

```
91.             dist[v] = dist[u] + graph[u][v];
92.             parent[v] = u;
93.         }
94.     }
95. }
96.
97.     printf("jalur terpendek dari %c ke %c: ", names[src],
names[dst]);
98.     printPath(parent, dst, names);
99.     printf("\ntotal beban minimum: %d\n", dist[dst]);
100. }
```

Output:

```
jalur terpendek dari S ke D: S -> A -> B -> D
total beban minimum: 3
```